DSAA3073 · Theories in Data Science

Boosting after zero training error

Learning Sheet · Lecture 9

Margins, optimization, and evidence about new data

Separate score size from vote agreement

LESSON BACKBONE

After the training labels are all correct, what can still change, and what evidence can justify continuing to boost?

We first separate score scale from relative vote agreement. We then read the distribution of normalized margins and a bound on population error. Finally, we examine the exact optimization step and the observations that receive large weights.

A training classifier can make no mistakes and still change when another vote is added. Its predictions on the observed rows may stay the same, while its scores, exponential loss, and predictions on other inputs change. Zero training error tells us one fact about the observed labels. It does not determine these other quantities.

Lecture 8 explained how AdaBoost reduces an exponential training loss and uses that loss to bound training classification error. The result applies to successful finite rounds under the stated weighted-error conditions. It does not say that training error equals exponential loss, or that a smaller training loss always gives a smaller error on new data.

We need a way to describe a vote more precisely than its number of correct labels. Before defining that quantity, consider a simple change that leaves the classifier unchanged.

Doubling the score does not change the prediction

Recall the real-valued ensemble score

$$F_T(x) = \sum_{t=1}^{T} \alpha_t h_t(x),$$

where each base classifier predicts $h_t(x) \in \{-1, +1\}$. The final label is the sign of the score. We continue to predict $+1$ when the score is zero. For a labeled observation, the **signed score** is $yF_T(x)$. A positive signed score gives a correct prediction; a negative one gives an error. At zero, correctness depends on the label and the tie rule.

Suppose we replace every coefficient by twice its value. The new score is $2F_T(x)$. Its sign is unchanged, so every predicted label is unchanged. Yet each nonzero signed score doubles. If we call a larger raw signed score "better agreement," this operation would seem to improve the vote without changing the relative influence of any classifier.

The exponential loss also responds to this scaling. At a positive signed score s, the loss is $\exp(-s)$. Replacing s by $2s$ makes the loss smaller. If every training signed score is positive, common positive scaling reduces every row loss while leaving every training prediction unchanged. This is one concrete reason that exponential loss can fall after training error reaches zero.

This observation does not make exponential loss uninformative. It tells us which question the loss answers. It measures the objective being optimized at the current score scale. To compare relative vote agreement, we must account for the total coefficient size.

Keep one set of observations and base predictions

The following small construction will let us compare these objects without fitting a new model each time. There are four distinct inputs with labels $(+1, +1, -1, -1)$. Four supplied base classifiers each make exactly one error, on a different row.

Row	y_i	$h_1(x_i)$	$h_2(x_i)$	$h_3(x_i)$	$h_4(x_i)$
1	$+1$	-1	$+1$	$+1$	$+1$
2	$+1$	$+1$	-1	$+1$	$+1$
3	-1	-1	-1	$+1$	-1
4	-1	-1	-1	-1	$+1$

We can form different votes from these same functions. The coefficients below are specified for comparison; they are not claimed to be successive AdaBoost rounds. Later, we will calculate one actual optimization step from a stated starting vote.

Worked example: A correct vote and its rescaling

Let vote A have coefficients $(0.45, 0.25, 0.20, 0.10)$. They sum to one. Its signed training scores are

$$(y_i F_A(x_i))_{i=1}^4 = (0.10, 0.50, 0.60, 0.80).$$

All four predictions are correct. Doubling all coefficients gives signed scores

$$(0.20, 1.00, 1.20, 1.60),$$

with exactly the same four labels. The coefficient sum has also doubled, from one to two. The ratio of each signed score to that sum is unchanged.

For row 1, the classifiers that predict its correct label carry total coefficient 0.55. The wrong classifier carries 0.45. Their difference is 0.10. Doubling the coefficients gives correct and incorrect totals 1.10 and 0.90. The raw difference becomes 0.20, but its fraction of the total remains 0.10.

Define a scale-independent signed margin

Definition: Normalized functional margin

For binary labels and base predictions, let $F_T(x) = \sum_{t=1}^T \alpha_t h_t(x)$. Assume the coefficient vector is nonzero, so

$$\|\alpha\|_1 = \sum_{t=1}^T |\alpha_t| > 0.$$

The normalized functional margin is

$$\rho(x, y) = \frac{y F_T(x)}{\|\alpha\|_1}.$$

It is the signed score divided by the total absolute coefficient size. For a training observation, write $\rho_i = \rho(x_i, y_i)$.

The denominator is positive, so normalization preserves the sign of the signed score. A positive margin gives a correct prediction; a negative margin gives an error. A zero margin still follows the declared tie rule. In particular, a positive label at a zero score is correctly predicted by our rule, although its margin is zero.

The range follows from the binary base predictions:

$$|yF_T(x)| = \left|\sum_{t=1}^{T} \alpha_t y h_t(x)\right| \le \sum_{t=1}^{T} |\alpha_t| = \|\alpha\|_1.$$

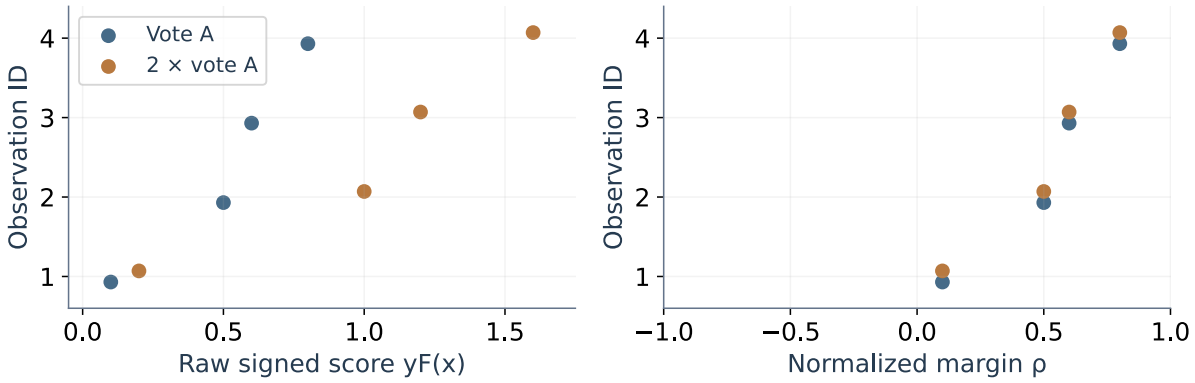
Thus $-1 \le \rho(x, y) \le 1$.

The inequality uses $|yh_t(x)| = 1$. It does not depend on the number of rounds or on whether the base predictions agree with each other.

For any common scale $c > 0$,

$$\frac{ycF_T(x)}{\|c\alpha\|_1} = \frac{cyF_T(x)}{c\|\alpha\|_1} = \rho(x, y).$$

Positive common scaling therefore changes the raw score but not this margin. Negative scaling is different: it reverses the score sign and can change the predicted labels. We do not include it in the invariance claim.



Each color uses the same four rows. Raw signed scores move when vote A is doubled. In the normalized panel, the paired points have the same horizontal position; the small vertical offset only keeps both colors visible.

A normalized margin is a measure of signed vote agreement. It is not a probability that a label is correct. It is also not a distance from the input to a decision boundary. Those interpretations require other objects and assumptions. Here the formula concerns classifier outputs and their coefficients, so it remains meaningful without assigning a geometry to the input space.

The nonzero-denominator condition matters. The initial AdaBoost score has all coefficients zero. At that point this normalized margin is undefined. Once there is a positive finite vote coefficient, the positive-coefficient normalization below is available. A zero score at a particular input is different from an entirely zero coefficient vector: the former has margin zero when the denominator is positive.

Turn a positive vote into a weighted average

For the positive-coefficient votes used in ordinary successful AdaBoost rounds, absolute values are unnecessary in the coefficient sum.

Put

$$A = \sum_{t=1}^{T} \alpha_t > 0, \quad \lambda_t = \frac{\alpha_t}{A}.$$

Then $\lambda_t \geq 0$ and $\sum_t \lambda_t = 1$. The normalized score is a weighted average of base predictions:

$$\frac{F_T(x)}{A} = \sum_{t=1}^{T} \lambda_t h_t(x).$$

This representation will identify the function class used by the margin bound.

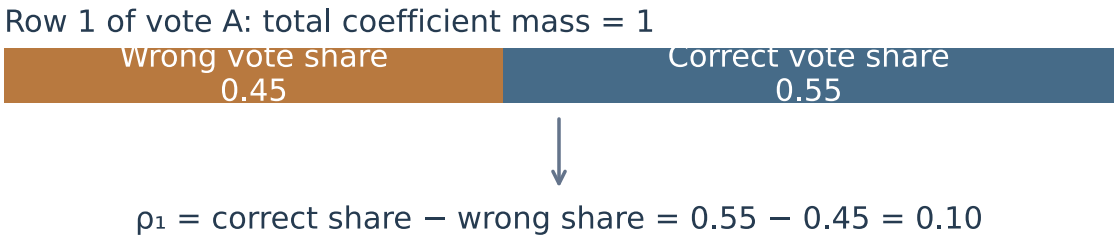
Definition: Convex hull of a base class

Let $\mathcal{H}$ be a class of functions from $\mathcal{X}$ to $\{-1, +1\}$. Its convex hull consists of all finite weighted averages

$$f(x) = \sum_{j=1}^{N} \lambda_j h_j(x), \quad h_j \in \mathcal{H}, \quad \lambda_j \geq 0, \quad \sum_{j=1}^{N} \lambda_j = 1.$$

The integer N may differ between averages. The coefficient sum is exactly one.

For a positive vote, $f = \frac{F_T}{A}$ belongs to this convex hull, and its margin is simply $yf(x)$. The normalization removes the arbitrary common scale while retaining the relative coefficients. It does not force all observations to have a large margin, or even a positive one.



For row 1, normalized correct vote share minus normalized wrong vote share gives the margin. Both shares are nonnegative and together sum to one.

More generally, let q be the normalized coefficient share of classifiers that are wrong on one observation. The correct share is $1 - q$, so the margin is $(1 - q) - q = 1 - 2q$. Equal shares give margin zero. A majority of correct vote weight gives a positive margin. This explains the endpoints: margin one means all positive vote weight is correct, and margin minus one means all of it is wrong.

The general margin definition also allows signed coefficients. Its absolute-value denominator still gives the range bound. However, signed coefficients do not immediately give a convex average of the original base class. We would need each negative contribution to be represented by the negated base function, with that function allowed in the class. We will explicitly introduce that closure condition when discussing signed coordinate directions. The population margin theorem below is applied directly to positive weighted averages.

Read the whole margin distribution

A vote can have a different margin on every training observation. The minimum margin describes only the smallest value. An average can hide a group of rows near zero. To see how much of the sample has small margins, count the observations below a chosen threshold.

Definition: Empirical margin distribution

For training margins $\rho_1, ..., \rho_m$, define the empirical low-margin proportion at threshold θ by

$$\hat{\rho}_\theta = \frac{1}{m} \sum_{i=1}^{m} \mathbb{1}[\rho_i \leq \theta].$$

As a function of θ, this is the empirical margin distribution. It is the fraction of observed margins at or below each threshold. An observation exactly at the threshold is included.

Here ρ_i is a margin value for one row. The quantity $\hat{\rho}_\theta$ is a proportion of rows. They use related notation but have different roles. For example, a threshold of 0.5 asks for the fraction whose margins are no greater than 0.5; the answer need not be 0.5.

Increasing the threshold cannot remove a counted row. The function is therefore nondecreasing. It changes only when the threshold reaches a training margin, so its graph is a step function. Each distinct margin contributes the fraction of rows that share that value.

At threshold zero, this proportion counts all nonpositive margins. It is an upper bound on the observed classification-error fraction under our tie convention. If every training margin is strictly positive, both quantities are zero. If there are zero margins, the two need not be equal.

Compare two correct votes on the same rows

Keep the four base classifiers and labels already given. Vote B has coefficients $(0.30, 0.25, 0.25, 0.20)$, which also sum to one. Because classifier h_j is wrong only on row j, each margin is one minus twice the corresponding coefficient.

Worked example: Same training error, different margin distributions

The supplied margins are

$$\rho^A = (0.10, 0.50, 0.60, 0.80), \quad \rho^B = (0.40, 0.50, 0.50, 0.60).$$

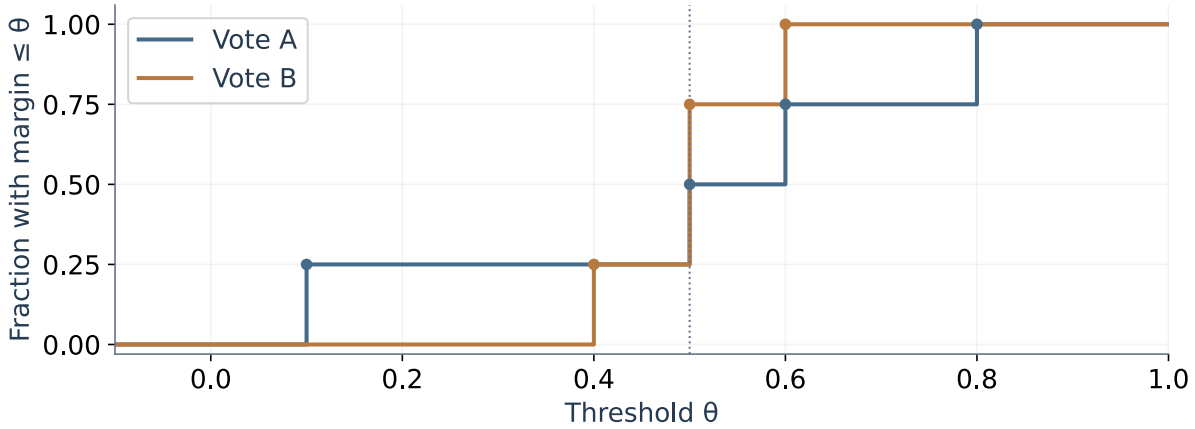
Both votes classify all four rows correctly. Vote B raises the minimum from 0.10 to 0.40. However, its largest margins are smaller: rows 3 and 4 move from $0.60, 0.80$ to $0.50, 0.60$.

The completed threshold counts are:

Threshold θ	Vote A: $\hat{\rho}_\theta$	Vote B: $\hat{\rho}_\theta$
0.20	0.25	0
0.40	0.25	0.25
0.50	0.50	0.75
0.60	0.75	1
0.80	1	1

At threshold 0.20, B has no low-margin observations, whereas A has one. At threshold 0.50, B has three counted observations, whereas A has two. Thus a larger minimum does not make B's low-margin proportion smaller at every threshold.

The equality convention is visible at $\theta = 0.50$. In B, the two margins equal to 0.50 count together with the margin 0.40. Omitting the equality cases would incorrectly report one quarter instead of three quarters.



The vertical position is the fraction at or below the horizontal threshold. Filled dots mark included jump values. B has less mass below a small threshold, but more mass at threshold 0.5*. The curves cross.*

Read the plot by selecting a threshold first, then moving vertically to a curve. The resulting height answers a specific sample question. It does not directly give population error. The two curves describe the same four training rows under two specified votes; no test observations are present.

There are two distinct comparisons here. Moving from A to $2A$ changes the scale and leaves the normalized distribution unchanged. Moving from A to B changes relative coefficients and therefore can change the distribution. Calling both changes "larger scores" would lose the distinction that normalization was designed to preserve.

What zero training error leaves undecided

All strictly positive margins give zero training error, regardless of how close some are to zero. The two constructed votes make this information loss visible. The zero-one count discards differences among correct predictions; the margin distribution records differences in relative vote agreement.

It would still be too strong to infer that a vote with larger margins must have lower population error. A training distribution is observed on a finite sample. To relate it to fresh observations, we need a statement about sampling and the class of possible votes. That statement must also account for how the vote was selected using the sample.

Likewise, these constructed states do not establish a typical AdaBoost trajectory. They show what zero training error does and does not determine. An algorithm might produce either state, neither state, or other states. The question about its actual update will require the exponential-loss objective and a selection rule, which we will examine after reading the population bound.

Read a margin bound with its conditions

The margin distribution is useful for generalization only when its relation to population behavior is controlled. The target quantity is the probability of a wrong label on a new draw from the same distribution that generated the training sample. It is not the empirical fraction on the current rows.

Let $\mathcal{D}$ be a distribution over labeled observations $(x, y) \in \mathcal{X} \times \{-1, +1\}$. For a classifier h, its **population classification risk** is

$$R_{\mathcal{D}}(h) = \Pr_{(x,y)\sim\mathcal{D}}(h(x) \neq y).$$

For a normalized vote f, write h_f for its sign classifier with $+1$ at a tie. Since a mistake requires a nonpositive signed score,

$$R_{\mathcal{D}}(h_f) \leq \Pr_{(x,y)\sim\mathcal{D}}(yf(x) \leq 0).$$

The right side also counts zero scores with positive labels, which our classifier predicts correctly. Keeping this inequality avoids changing the tie convention midway through the argument.

Identify the class and the random object

Fix a binary base class $\mathcal{H}$ before drawing the sample. Recall that its VC dimension is the largest number of inputs on which the class can realize every binary labeling. We denote it by d. It measures the capacity of the **base class**, not the number of classifiers used in one fitted vote.

The sample $S = ((x_1, y_1), ..., (x_m, y_m))$ consists of independent draws from $\mathcal{D}$. A learning algorithm may use this sample to choose functions from $\mathcal{H}$ and their vote weights. We therefore want a guarantee that covers every normalized vote in the class, rather than one vote fixed independently of the data.

There are two more parameters. The confidence parameter $\delta \in (0, 1)$ controls the allowed probability that the sample-level guarantee fails. The positive margin threshold θ determines which observed rows count as low-margin. These parameters do different jobs: changing the threshold changes the event counted on each row; changing confidence changes how strong a sampling guarantee we request.

Result: A uniform population-error margin bound

Fix a class $\mathcal{H}$ of binary base classifiers with VC dimension $d \geq 1$. Let S contain $m \geq 2d$ independent labeled draws from $\mathcal{D}$. Choose $\delta \in (0, 1)$.

Write $\text{conv}(\mathcal{H})$ for the convex hull of $\mathcal{H}$ defined above.

There is a universal constant $C > 0$ such that, with probability at least $1 - \delta$ over the sample, the following holds simultaneously for every $f \in \text{conv}(\mathcal{H})$ and every $\theta \in (0, 1]$:

$$R_{\mathcal{D}}(h_f) \leq \Pr_{(x,y)\sim\mathcal{D}}(yf(x) \leq 0) \leq \hat{\rho}_\theta + C\sqrt{\frac{d\log^2\left(\frac{m}{d}\right)}{m\theta^2} + \frac{\log\left(\frac{1}{\delta}\right)}{m}}.$$

Here $\hat{\rho}_\theta = m^{-1}\sum_{i=1}^{m} \mathbb{1}[y_i f(x_i) \leq \theta]$, and h_f predicts $+1$ when $f(x) = 0$.

The constant is independent of the base class, vote, sample size, threshold and confidence parameter. Its numerical value is not specified here. The result gives a dependence on these quantities; it is not a numerical error certificate obtained by setting $C = 1$.

The restriction $m \geq 2d$ keeps us in a nondegenerate range of this expression. The theorem does not assume the labels are perfectly realizable by the base class or by the vote. Its empirical low-margin term may be large. It can therefore describe votes with training mistakes as well as votes with zero training error.

Read the two terms before using the bound

The empirical term $\hat{\rho}_\theta$ is observable once the vote and training sample are known. It includes every wrong or tied row, and it also includes correct rows whose normalized margin is positive but no greater than θ. Counting these additional rows is the price of asking for positive vote agreement rather than correctness alone.

The second term accounts for complexity and sampling uncertainty. For fixed m, d, δ, a smaller threshold increases the inverse-threshold contribution. A tiny positive margin gives little separation from the zero-score boundary in this normalized score scale. The theorem pays for that small threshold with a larger penalty.

The class-capacity factor uses d, while the confidence factor uses $\log\left(\frac{1}{\delta}\right)$. Asking for higher confidence means smaller δ, which increases this term. The sample size appears in the denominator, and it also appears inside the logarithm. We must read the whole expression rather than treating only one factor as a prediction of observed performance.

The theorem is one probability statement about the sample. On its successful event, it covers **all** normalized votes and **all** allowed thresholds at once. The vote can therefore be selected from training data without invalidating this particular uniform guarantee. Comparing allowed thresholds using the displayed bound is also covered by the same event. This is different from a theorem stated only for a single threshold fixed before sampling.

Uniformity does not remove the theorem's other conditions. The base class must be the fixed class whose capacity is accounted for. If a procedure also searches over different base classes, the displayed statement cannot simply ignore that search and quote the most favorable class complexity afterward.

Compare votes at a fixed threshold

> **Worked example: What a common penalty lets us compare**
>
> Take the same sample size, base class and confidence level for votes A and B. At threshold $\theta = 0.20$, their empirical low-margin terms are 0.25 and 0. The complexity penalty is the same for both because none of its inputs changed.
>
> Thus the displayed upper bound for B is smaller by 0.25 at this threshold. This compares two upper bounds. It does not establish that B's actual population error is smaller by 0.25, or even that the two actual errors have that ordering.
>
> At $\theta = 0.50$, the empirical terms are 0.50 and 0.75, so the ordering of these displayed bounds reverses. The example shows why a statement about "better margins" should name the distribution or threshold being compared.

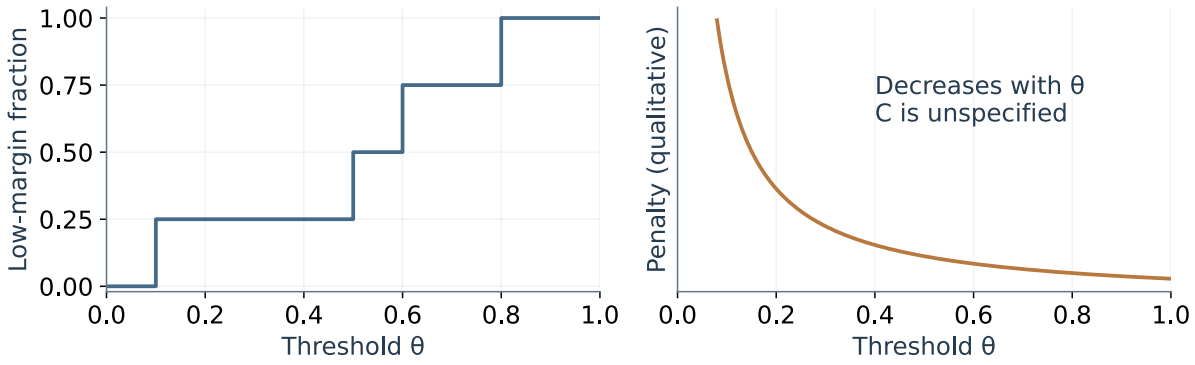
The four-row construction supplies empirical terms only. It is not being used to certify a small error percentage: we have not supplied a numerical universal constant or a base-class/

sample regime yielding a useful numerical penalty. The comparison of equal penalties remains meaningful without pretending that the bound is tight.

An upper bound above one can also be valid but uninformative, since a classification risk is at most one. A bound need not closely match observed error to be mathematically correct. Conversely, a small observed test error does not verify the assumptions of a theorem by itself.

Changing the threshold changes both sides of the tradeoff

As θ increases, more training margins may be counted, so the empirical term cannot decrease. The complexity penalty decreases because θ^2 is in the denominator under the square root. A useful threshold balances these two effects.



Left: the actual empirical low-margin curve for vote A. Right: only the decreasing shape of the complexity penalty as the threshold grows. Its vertical scale is deliberately unspecified because C is unspecified; the panels do not provide numerical values to add.

A threshold just below the smallest margin gives empirical term zero, but it may produce a large penalty. A threshold near one gives a smaller penalty, but usually counts many rows. Therefore neither "always choose the smallest threshold" nor "always choose the largest threshold" follows from the theorem.

The simultaneous statement permits threshold comparison within the theorem. It does not turn margin-based tuning into an independent measurement of future error. If we want an empirical estimate of the performance of a selected predictor, the selection-and-evaluation distinction from Lecture 7 still applies.

Why different upper bounds need not disagree

A capacity bound can describe a broad class of ensembles with a specified number of terms. A margin bound uses the normalized vote and the empirical frequency of low margins. These analyses retain different information about the same prediction problem.

Suppose one valid upper bound grows as the allowed ensemble class becomes larger, while another becomes smaller for a vote with fewer low-margin observations. There is no contradiction. Neither expression is an equality for the actual risk. A quantity can have several valid upper bounds, and those bounds can change in different directions.

The margin result is connected to uniform-convergence reasoning for binary base classes. Its refinement uses the distribution of normalized agreement rather than only the number of terms in a vote. We do not need a new proof technique to read its conclusion here. In particular, a coefficient norm constraint must not be mistaken for a guarantee that every point has margin at least θ.

The number of boosting rounds does not appear explicitly in the displayed penalty. More rounds can still change the fitted normalized vote and its empirical margin distribution. The theorem says what follows **if** those quantities give a favorable right-hand side; it does not say that another AdaBoost round will improve them.

Zero training error records signs, while normalized margins record relative agreement. The theorem relates the observed margins to population error under its sample and class conditions. It leaves a different question unanswered: what does AdaBoost actually choose to decrease at each round?

When does AdaBoost choose the steepest coordinate?

The margin bound applies to every normalized vote in its class. It does not specify an algorithm for finding a vote. AdaBoost makes its update by minimizing an exponential training objective. To understand that update, return to the raw score and its coefficients; the normalized margin bound is not a premise of this optimization argument.

Lecture 8 held one classifier fixed and chose the best coefficient along that classifier. That is a line-search calculation. A stronger statement would also say that the algorithm chooses the best available coordinate direction. These are separate choices, and they need separate conditions.

Represent the score by a fixed coefficient vector

Fix a finite dictionary of binary base functions

$$\mathcal{H} = \{h_1, ..., h_N\}.$$

The index j now names a dictionary function. It is not a round index. If a later round reuses the same function, we add to its existing coefficient rather than introducing a different function merely because time has advanced.

Definition: Exponential loss in coefficient space

For a coefficient vector $\beta \in \mathbb{R}^N$, define

$$F_\beta(x) = \sum_{j=1}^{N} \beta_j h_j(x),$$

and the empirical objective

$$J(\beta) = \frac{1}{m} \sum_{i=1}^{m} \exp\big(-y_i F_\beta(x_i)\big).$$

The functions, training inputs and labels are fixed. The variable being optimized is the coefficient vector β.

Each summand is the exponential of a linear function of the coefficients. Hence J is convex and differentiable. These properties support the one-dimensional minimization we will use. They do not say that one update reaches a global optimum, or that a finite minimizer must exist. Exponential loss can keep decreasing along a separating direction as coefficients grow.

Let e_j be the coordinate vector with a one in position j and zeros elsewhere. A coordinate update replaces β by $\beta + ae_j$.

The resulting score is

$F_{\beta+ae_j}(x) = F_\beta(x) + ah_j(x)$.

Only one coefficient changes. The corresponding direction in score space is the base function h_j. Thus coefficient movement and addition of a classifier are two descriptions of the same operation.

The coordinates are not input features. For example, moving the coefficient of h_4 does not move a training observation along a fourth feature axis. It changes how much the existing function h_4 contributes to every score.

Reconstruct the current row weights

At a fixed current coefficient vector, put

$$s_i = y_i F_\beta(x_i), \quad D(i) = \frac{\exp(-s_i)}{\sum_{k=1}^m \exp(-s_k)}.$$

Because J is a mean over m rows, the common denominator is $mJ(\beta)$. Consequently,

$$\exp(-s_i) = mJ(\beta)D(i).$$

These are the normalized exponential row weights from Lecture 8. The total loss and the mean loss differ by the factor m; retaining that factor makes the derivative calculation below transparent.

For each candidate function, define its error under these same weights:

$$\varepsilon_j = \sum_{i=1}^m D(i)\mathbb{1}\big[h_j(x_i) \neq y_i\big].$$

All candidates are compared at the **same current score** and therefore the same distribution D. We do not update the weights separately before measuring each candidate's derivative.

The signed prediction $y_i h_j(x_i)$ is $+1$ on a correct base prediction and -1 on an incorrect one. Its weighted mean is therefore

$$\sum_{i=1}^m D(i)y_i h_j(x_i) = \big(1-\varepsilon_j\big) - \varepsilon_j = 1 - 2\varepsilon_j.$$

This quantity measures how much correct weighted mass exceeds incorrect weighted mass for that candidate. It will determine the direction of its initial loss change.

Derive the slope for every candidate

The derivative with respect to coefficient β_j is obtained by differentiating the finite sum:

$$\frac{\partial J}{\partial \beta_j} = -\frac{1}{m}\sum_{i=1}^m y_i h_j(x_i)\exp\big(-y_i F_\beta(x_i)\big).$$

Substitute the normalized row-loss expression:

$$\frac{\partial J}{\partial \beta_j} = -\frac{1}{m}\sum_{i=1}^m y_i h_j(x_i) mJ(\beta)D(i).$$

The factors m cancel, and $J(\beta)$ is common to all terms. Hence

$$\frac{\partial J}{\partial \beta_j} = -J(\beta) \sum_{i=1}^{m} D(i) y_i h_j(x_i) = -J(\beta)\big(1 - 2\varepsilon_j\big).$$

Since $J(\beta) > 0$, an error below one half makes this derivative negative. A small positive coefficient added to that classifier decreases the loss. This is the positive-edge interpretation already available in Lecture 8.

To compare directions, we need more than the sign. A candidate with error 0.40 has factor $1 - 2\varepsilon = 0.20$. A candidate with error 0.10 has factor 0.80. At the same current score, the second gives a negative derivative four times as large in magnitude. Either candidate has positive edge, but they do not give the same initial descent.

The comparison concerns a unit change in the stated coefficient coordinates. It does not compare arbitrary rescalings of the base functions. Our binary dictionary fixes their output scale, which makes the errors and derivative magnitudes comparable.

Account for both signs of a coordinate

Coordinate descent may increase or decrease a coefficient. If increasing β_j has a positive derivative, decreasing it has a negative derivative. A search over only positive additions of available functions need not include that second direction.

Definition: Sign-closed dictionary and greedy coordinate selection

A dictionary is **closed under negation** if $h \in \mathcal{H}$ implies $-h \in \mathcal{H}$.

For a differentiable objective in fixed coordinates, greedy signed-coordinate selection chooses a partial derivative with the largest absolute value, then uses the sign that decreases the objective. Equivalently, it chooses the most negative directional derivative among all positive and negative coordinate directions.

For binary labels and predictions, negating a base function exchanges its correct and incorrect rows. Thus

$$\varepsilon(-h) = 1 - \varepsilon(h).$$

Adding $a(-h)$ with $a > 0$ gives the same score change as adding $-ah$. If the dictionary contains both functions, positive additions over the dictionary represent both coordinate signs.

This condition connects the error-minimization rule to the absolute-derivative rule. Because both signs are available, the smallest weighted error is at most one half. It gives the largest nonnegative value of $1 - 2\varepsilon$. By negation closure, every negative value of this weighted signed mean has its positive counterpart available. When this value is positive, the smallest error over the sign-closed dictionary gives the descending orientation of the largest absolute partial derivative.

Without this condition, minimizing error over the available positive additions only establishes the best of those additions. For instance, candidate errors 0.40 and 0.90 correspond to weighted signed means 0.20 and -0.80. The first is better as a positive addition, but the negative direction of the second has a larger descent magnitude. If that negated function is allowed, its error is 0.10. This small comparison explains why the sign condition belongs in the stronger theorem.

Result: Exact greedy coordinate interpretation of an AdaBoost round

Use a fixed finite dictionary of binary base functions that is closed under negation. At the current score, use the normalized exponential row weights.

Assume the base learner finds a function with the exact minimum weighted error over this dictionary. Resolve any ties consistently. Suppose its error satisfies $0 < \varepsilon < \frac{1}{2}$, and choose its coefficient by exact line search.

Then the update is a greedy signed-coordinate descent step on the empirical exponential objective: it selects a descending coordinate of maximum derivative magnitude and minimizes the objective along that coordinate.

Finiteness ensures that the minimum over the dictionary is attained. Exact selection is a property of the fitting rule, not a consequence of the coefficient formula. A base learner that returns any positive-edge function supplies a descent direction, but the theorem cannot call that direction the steepest one without the exact-search condition.

Match the line-search coefficient

After selecting a function h, keep it and the current score fixed. The coefficient a is now the only variable. Using the current weights, the same factorization as in Lecture 8 gives

$$\frac{J(\beta + ae_j)}{J(\beta)} = \sum_{i=1}^{m} D(i) \exp(-a y_i h(x_i)) = (1-\varepsilon)\exp(-a) + \varepsilon \exp(a).$$

The correctly predicted weighted mass contributes the first term, and the incorrectly predicted mass contributes the second.

For $0 < \varepsilon < \frac{1}{2}$, the unique finite minimum occurs at

$$a = \frac{1}{2}\log\left(\frac{1-\varepsilon}{\varepsilon}\right).$$

To recall why, equate the two terms in the derivative:

$$-(1-\varepsilon)\exp(-a) + \varepsilon\exp(a) = 0 \quad \Rightarrow \quad \exp(2a) = \frac{1-\varepsilon}{\varepsilon}.$$

The second derivative is

$$(1-\varepsilon)\exp(-a) + \varepsilon\exp(a) > 0,$$

so this stationary point is indeed the unique minimum along the chosen line. This recovers the AdaBoost coefficient, after direction selection has already been justified.

The two choices now match. Exact weighted-error search in a sign-closed binary dictionary selects the greedy descending direction. Exact line search gives the AdaBoost coefficient along that direction. This establishes the optimization theorem without using the margin bound or a claim about test performance.

At zero weighted error, the ratio is $\exp(-a)$ and has no finite minimizer. The separate perfect-training-classifier stopping branch from Lecture 8 applies. At error one half, the minimizing coefficient is zero and the score does not change. These cases are excluded from the finite positive-step statement, rather than being assigned an infinite or undefined coefficient inside it.

Follow one exact step from the supplied vote

Return to vote A and the four fixed base classifiers. For this coordinate calculation, allow their negatives as well, so the dictionary has eight signed candidates. The current score is still vote A; this is not a new dataset.

Worked example: Choose a direction before choosing its length

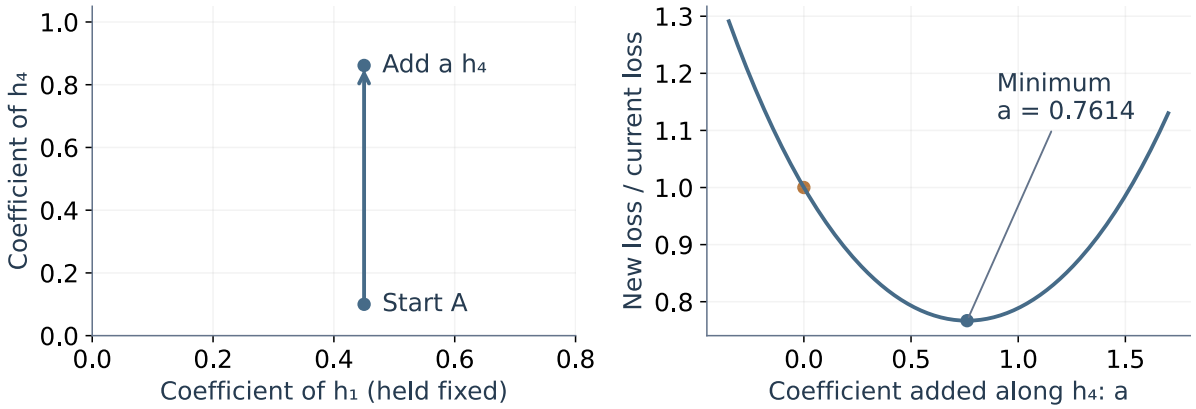
Vote A has signed scores $(0.10, 0.50, 0.60, 0.80)$. Normalize their exponential losses to get the current row weights. Each positive function h_j is wrong only on row j, so its weighted error equals that row's mass.

The supplied candidate errors are approximately:

Candidate	Weighted error	Derivative divided by $J(\beta)$
h_1	0.36056	−0.27887
h_2	0.24169	−0.51661
h_3	0.21869	−0.56261
h_4	0.17905	−0.64190

For a negative candidate, replace its error by one minus the displayed error and reverse the derivative sign. The exact minimum is therefore attained by h_4. Its positive addition gives the largest initial decrease per unit coefficient among the signed candidates.

Using the unrounded values, the selected coefficient is about 0.76140. The coefficient of h_4 changes from 0.10 to 0.86140, while the other three coefficients stay fixed. The exact line-search loss ratio is about 0.76679; the mean loss changes from about 0.62738 to 0.48107.



Both panels describe the same step from vote A. Left: only the coefficient of h_4 changes; the coefficient of h_1 is shown fixed at 0.45*. Right: the loss ratio along that selected direction reaches its minimum at the added coefficient* $a \approx 0.76140$.

The left panel separates a coordinate direction from its step length. The right panel answers the remaining one-variable question once h_4 is fixed. The other coefficient values are held constant in both panels, even though only two coefficient axes are displayed.

This step minimizes the loss along one line. It need not minimize over all coefficient vectors at once. Nor does choosing the best initial coordinate slope imply a new theorem about the

normalized margin distribution. We now have enough information to compare those quantities directly, after examining how the same objective emphasizes persistent errors.

Follow a row with large loss and choose the evidence

The exponential weights were useful because they converted candidate errors into derivatives. They also determine which training observations receive attention. A row with a negative signed score has exponential loss above one. A row with a large positive signed score has a much smaller loss. This difference can make a persistently difficult row dominate the next weighted fitting problem.

Compare relative masses before normalizing

For current signed scores $s_i = y_i F(x_i)$, the normalized row mass is

$$D(i) = \frac{\exp(-s_i)}{\sum_k \exp(-s_k)}.$$

For any two rows, the common denominator cancels:

$$\frac{D(i)}{D(j)} = \frac{\exp(-s_i)}{\exp(-s_j)} = \exp(s_j - s_i).$$

Thus the relative emphasis depends exponentially on the difference between their signed scores. We can understand which row dominates without confusing the normalizing denominator with a vote coefficient.

For example, if one signed score is -1.2 and another is 1.6, their mass ratio is $\exp(2.8) \approx 16.44$. Doubling both scores changes the ratio to $\exp(5.6) \approx 270.43$. The prediction signs have not changed, but the relative training emphasis has changed greatly.

The derivative magnitude of the per-row exponential loss with respect to its signed score is also $\exp(-s)$. Hence the same row that carries a large loss has a large effect on the loss derivative. This explains why the weighting mechanism concentrates on low signed scores. It is not a statement that every such row has a wrong label.

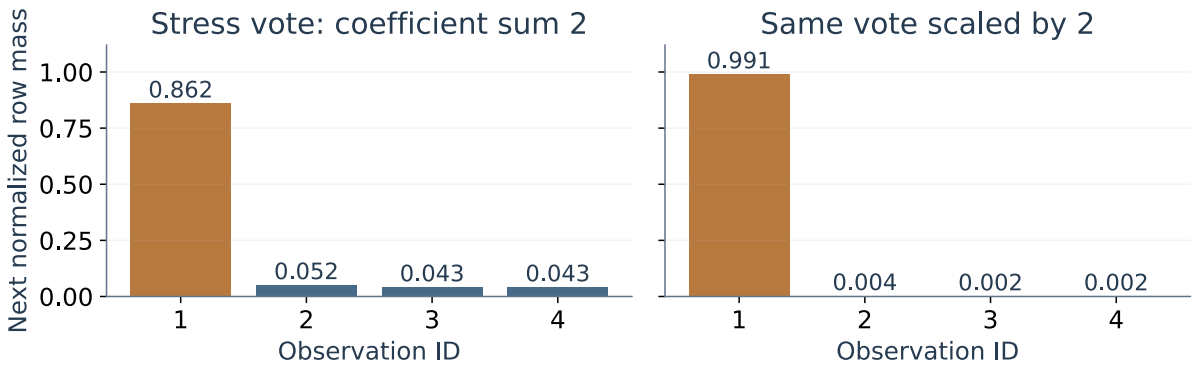
A constructed persistent-error comparison

Keep the same labels and four base prediction columns. Change the relative coefficients to $(0.80, 0.10, 0.05, 0.05)$ and first multiply them by two. This produces a deliberately constructed stress vote, not the next round after vote A.

> **Worked example: One negative-score row receives most of the mass**
>
> With coefficient sum two, the signed scores are
>
> $$(-1.2, 1.6, 1.8, 1.8).$$
>
> Row 1 is wrong and the other rows are correct. The next normalized row masses are approximately
>
> $$(0.86178, 0.05241, 0.04291, 0.04291).$$
>
> Now double all coefficients of this stress vote. The signed scores become $(-2.4, 3.2, 3.6, 3.6)$.

The labels and normalized margins are unchanged, but the next masses become

$$(0.99142, 0.00367, 0.00246, 0.00246).$$



Amber always identifies row 1. Both panels use the same probability scale and row IDs. Common score scaling preserves the normalized margin distribution but increases the mass assigned to the persistent negative-score row.

This comparison isolates a mechanism. If an observation remains in strong disagreement with the current score while other scores become strongly positive, its relative mass can become very large. In a real training problem, a wrong label is one possible reason for persistent disagreement. Difficult structure or a restricted base class can also cause it. The weights alone cannot identify which explanation is correct.

The construction is not a claim that AdaBoost actually performs this common scaling or necessarily follows these two states. It shows how exponential normalization responds when those signed-score differences occur. It also explains why clean-data intuition cannot be turned into a guarantee that more rounds help every dataset.

The margin bound remains a statement about the distribution generating the labels being evaluated. It does not automatically change its target to an unobserved "clean" label when some training labels may be noisy. Diagnosing label quality and evaluating predictions require evidence beyond the magnitude of one training weight.

Lower exponential loss can reduce the minimum normalized margin

Return to the **actual single coordinate step** calculated from vote A. This is a different comparison from the constructed stress scaling. Its labels, dictionary, chosen function and coefficient update were specified above.

Worked example: Compare the objective with the normalized margins

Before the step, vote A has normalized margins

$$(0.10, 0.50, 0.60, 0.80).$$

After adding the exact positive coefficient to h_4, the normalized margins are approximately

$$(0.48904, 0.71613, 0.77291, 0.02192).$$

All four remain positive, so training error stays zero. The exponential loss decreases because the coefficient was chosen by exact line search. Yet row 4's normalized margin falls from 0.80 to about 0.02192. The minimum margin also falls, from 0.10 to about 0.02192.

There is no conflict with the optimization proof. The chosen classifier h_4 is correct on rows 1–3 and wrong on row 4. Adding its vote helps the first three signed scores and reduces the fourth. At the selected coefficient, the total exponential loss is smaller. The objective permits one row to become worse while the aggregate improves.

Normalization also changes because the total coefficient sum increases. Therefore the optimized exponential objective and the minimum normalized margin are different functions of the coefficients. An exact greedy step for one need not increase the other. This supplied example rules out a claim that every AdaBoost round must raise the minimum normalized margin.

The population bound is consistent with this distinction. It evaluates empirical low-margin mass at a threshold, together with a complexity penalty. It does not assert that the training algorithm monotonically improves that mass at all thresholds. An algorithmic update, an optimization guarantee and a population bound can all be valid without being the same statement.

Match the evidence to the question

Zero training error leaves many changes possible. Raw scores can rescale, relative coefficients and margin distributions can change, exponential loss can decrease, and training weights can concentrate. Each describes a different aspect of the fitted vote. None alone determines the error on fresh data.

Question	Evidence that answers it
Which labels are correct on the training rows?	The observed sign predictions and labels, with the stated tie rule.
What objective does this round decrease?	The exponential-loss calculation; exact coordinate selection needs the dictionary/search assumptions.
What bounds training error after successful rounds?	The conditional loss-product and edge bound from Lecture 8.
What population guarantee follows from margins?	The uniform margin theorem, with its base class, sample, threshold, confidence and normalization conditions.
How well does the selected procedure predict new observations?	Independent evaluation of the completed selection-and-fitting procedure.

Use validation evidence to choose a stopping round or other fitting settings. If those choices depend on validation results, the same results have participated in selection. Separate test evidence or an appropriate outer evaluation procedure is needed to evaluate the completed choice. Repeatedly inspecting a test score to decide when to stop would give that score a selection role.

Zero training error is therefore not a universal instruction to stop or to continue. The appropriate choice depends on the prediction objective and the selection evidence. The margin and optimization analyses explain quantities worth examining, while preserving the distinction between an explanation of training behavior and an estimate of new-data performance.

Standard language to carry forward

Term or object	Meaning in this lesson
Signed score $yF(x)$	The raw score multiplied by the label; its sign records correctness away from ties.
Normalized functional margin ρ	Signed score divided by the nonzero total absolute coefficient size; invariant under common positive scaling.
Convex average	A finite nonnegative combination of base functions whose coefficients sum exactly to one.
Empirical margin distribution	For each threshold, the fraction of training margins at or below it.
Uniform margin bound	One high-probability event controls every allowed normalized vote and threshold under the stated sample and class conditions.
Greedy signed-coordinate selection	Choose a coefficient direction of maximum derivative magnitude with its descending sign.
Exact line search	Minimize the objective along the already selected direction.

A positive rescaling leaves normalized vote agreement unchanged. A new relative vote can change that agreement. Exact AdaBoost selection and line search describe how the exponential objective changes under their assumptions. The margin theorem describes a population guarantee under a different set of assumptions. Keeping these objects and conditions explicit lets us interpret continued boosting without turning zero training error, large margins or low loss into an unsupported promise about new data.